DEFENSE_PREP.md - how to answer the jury (read this the night before)

Companion to `DEFENSE.md` (claims and evidence table) and `SYNTHIE.md` (what a synthie is). This file answers the questions a jury is likely to ask, explains every term in plain words, and says plainly where the project is weak. **If a sentence here is not backed by a file in the repo or a source in section 11, treat it as opinion.**

How this was researched (2026-10-29): the session's web-search tool was unavailable, so facts were pulled directly from arXiv's API (titles, dates, abstracts), Wikipedia's REST summaries and GitHub READMEs. What was **verified** that way is marked **[V]**. Anything from memory that was **not re-checked** is marked **[M]**: verify those before putting them on a slide.

---

0. Fix these three things BEFORE the defense (the poster does not match the project)

The poster ("Beyond MCPs: Self-Healing & Adversarially Resilient GUI Agents") promises more than the repo does. A careful juror will notice. Pick one answer for each and rehearse it.

#	the poster says	the project actually did	what to do
1	Phase 2 "Adversarial Resilience": **"Environment & Fingerprint Stealth"**, **"Human-Plausible Touch Telemetry"**, "DOM Rotation & Tree Suppression"	Scope was locked (plan section 2.4): behavioural-fingerprint survival is **evasion research and is out of scope; do not present it, do not implement it**. We *measure* how a site's protection treats an agent (`fyp.mjs protect`: Cloudflare Bot Fight Mode 9/9 completed, managed challenge 0/9) and report mechanisms and rates. DOM rotation / tree suppression exist as **test breaks** (B1, B8) that we survive or fail honestly on, not as evasion techniques	**Edit the poster** (suggested wording below) or say: "Phase 2 describes the *threat landscape* the agent faces. We implemented the resilience to UI change and the honest failure; we measured, and deliberately did not build, evasion." Never claim stealth or telemetry spoofing
2	"Running an online store on **Daraz/OLX**"; "Legacy Local Apps (Daraz, OLX)"	We tested on **our own mock Instagram web app** and **stock Android Settings** on an emulator. Daraz and OLX were **never tested** (OLX auto-selling was explicitly out of scope, plan 2.3)	Say Daraz/OLX are the **motivating target class**, not an evaluated one. Do not say "works on Daraz"

#	the poster says	the project actually did	what to do
3	"**0/9 Baseline vs 9/9 Self-Healed under UI chaos**"	True for single breaks such as B1 (the recorded script stops at step 1 with 0/9; the ladder completes 9/9). **Not true for "chaos" in general**: with all seven breaks together, or with B8 (a control that cannot be reached), the ladder stops honestly at step 1	Say "**under each single UI break we tried, and combinations up to four at once**"; show the N=10 table (`results/published/rules_full_n10/`). Say the all-seven case stops honestly *by design*

Suggested replacement for the poster's Phase 2 box: "*Phase 2 (measured, not evaded): how hostile UIs and protection layers treat an agent. We report suppression mechanisms and rates and fail honestly when a control is unreachable; we do not build bypasses.*" Also soften the one-sentence claim to: "*verified, self-healing GUI automation for users who cannot take over a failed step.*"

---

1. What everything is, in plain words (glossary)

term	plain meaning
GUI agent	software that operates an app like a person: looks at the screen, decides, taps, types
intent	*what* is wanted ("open the create-post flow"), never *which button*. 9 intents make the create-post flow
locator	a recipe for finding one control (an id, a label, a position). Recorded scripts store one locator per step and break when the app changes
harness (`fyp-mvp/harness.js`)	**our core product**: the code injected into the app page that holds the intents, finds controls, taps them, checks the result and logs everything. "The hands"
ladder / route	the ordered list of ways to find a control: `id`, `label`, `text`, `scoped` (inside a container), `learned` (remembered), `semantic` (ask a model), `vision` (position). A route counts only if it points at **exactly one** control
success predicate / postcondition	a check on the **next screen** that says whether the step achieved its goal ("the caption screen is showing"). It is independent of how the control was found. *This is how we know a step worked*
honest failure	if no route achieves the goal, stop and say which step and why in plain first-person words, ending "You may need to do this step yourself"
wrong action	a tap that did not achieve its step. **Silent** wrong action = the agent did not notice and carried on (the dangerous kind)
baseline	the old way: one recorded locator per step, no checking. Our comparison arm
arms	`baseline` (recorded script), `healed` (our rules only), `llm-only` (a model's first choice, nothing checked), `model-verified` (model + check, no rules), `hybrid` (rules + model + check = **our full system**)

term	plain meaning
breaks B1-B11	controlled UI changes we inject on purpose: B1 rename ids, B2 strip labels, B3 reverse layout, B4 duplicate a control, B5 nest in a scroller, B6 bare icon, B7 cover with an overlay, B8 hide completely, B9 reword text, B10 move the control, B11 plant instructions in labels
route memory / distillation	remembering which route worked, and turning a model's answer into a cheap locator so the model is not asked again
accessibility tree	the structured list of controls (names, roles, states) that screen readers use. On Android it is what `uiautomator dump` returns; on the web, the DOM + ARIA. Our *measuring instrument* is the same data a blind user's screen reader gets
UiAutomator / adb	Android's tool for dumping that tree and sending taps; `adb` is the command-line bridge to a phone or emulator
emulator `kenna`	a simulated Android 34 phone on this laptop (SDK at `D:\AndroidSdk`)
MCP (Model Context Protocol)	an open standard for giving AI systems tools and data through a uniform interface [V: introduced by Anthropic Nov 2024, adopted by OpenAI and Google DeepMind, donated to the Agentic AI Foundation under the Linux Foundation in Dec 2025 - Wikipedia 2026-10-29]. Where a service offers MCP/API you should use it; this project is about the apps that do not
LLM / model rung	a language model asked "which of these on-screen controls means this goal?"; it may only answer with an index into that list
OpenRouter, `qwen/qwen3.7-flash`	the service and the model used for the real runs (text + image input, about $0.03 / $0.13 per million tokens [V: OpenRouter model listing])
offline stub	a no-cost word-overlap stand-in used only to test plumbing. **Never quote it as a model result**
temperature 0, cache	the model is asked for its most likely answer and answers are stored on disk, so a re-run is identical. Repetitions therefore show variation in the *app and harness*, not in the model
Wilson interval	a 95% confidence range for a pass rate. With 10 trials, 10/10 is "72-100%", not "100%"
Momo	the *assistant* governed by `agent/MOMO.md`: it says goals, never locators; it tells the user the truth about the result; it never blind-retries; it confirms before publishing for someone else
Kenna	two meanings, which confuses people: (a) the **assistant** the user talks to, (b) the **repo folder** `Kenna/`, which holds the *tool* the assistant uses
synthie (e.g. Hematite)	a long-running agent on its own host with a chat identity, a typed tool set and logged actions; it is the kind of agent that *uses* this tool. **Most of what we know about synthies is a synthie's own description**; see `SYNTHIE.md` section 0 and confirm with its owner before putting claims on a slide

How the pieces fit:

```
user  -->  assistant (Momo / Kenna / a synthie)  --goal-->  tool (agent/fyp.mjs)
-->  harness (the ladder)  -->  the app
  ^
|
  |<-- plain-words result: ok / "I could not do step X because ..." <-- success
check on the next screen <--+
                                        (a model may be asked ONLY inside the
ladder, only for an index, and is checked like any other route)
```

---

2. THE core question: how does it know it did the right step?

2.1 The mechanism

Every intent carries a **success predicate**: a small function that reads the screen *after* the action. It does not care which route found the control. The nine predicates (from `harness.js`):

intent	the check after the tap
`post.open`	the gallery screen is showing
`gallery.select`	the selected photo's signature changed (the preview followed it)
`compose.next`	the caption screen is showing
`meta.advance`	the "more options" screen is showing
`meta.back`	back on the caption screen
`caption.type`	the caption field's value equals the text we typed
`setting.toggle`	the switch reads as on (the real `<input>` state, not the wrapper)
`share.post`	back on the feed **and the new caption is in the feed**
`feed.assert`	the published caption is present in the feed (an end-state check that does not depend on any locator)

Rules that make this honest: a tap that raised no error is **never** counted as success; the predicate is the only thing that decides. A route is acted on only if it matches **exactly one** control. If the predicate fails, the attempt is logged `(no effect)`, the same control is not hammered with the same route, and the ladder moves on.

2.2 "What if the rules and the model disagree?"

There is no voting and no trust ordering between them; the screen decides. The procedure, in plain steps:

1. The routes run in a fixed order: id, label, text, scoped, learned, semantic (model), position. The first route that points at exactly one control proposes it.
2. The harness taps it and runs the predicate.

3. If the predicate holds: done, and the log says which route won. If not: that route is marked wrong for this step, and the next route proposes.
4. The model is only reached when the rules found nothing unique or their pick failed. If it names a different control, that control is tapped and checked the same way. "Disagreement" is therefore resolved by **outcome**, and it costs a *wasted action*, which we count (hybrid: 0-8 wasted taps per run in the qualification; a wrong tap that the check catches is time lost, not a wrong result).
5. If every route fails: honest failure.

A real example (reskin B1+B9+B10, ids rotted, words changed, control moved): the rules found nothing for most steps, so the model proposed (`semantic[0]`) and each pick was verified: 9/9 completed with 7 model calls. With the rules alone the same run stops at step 3.

Honest limits of this answer (say them first):

- *False positive:* a predicate that is too weak can pass for a wrong action. We hit this: `share.post` originally only checked "back on the feed", which any tap that abandoned the draft also satisfies. It now also requires the caption in the feed, and `feed.assert` is an independent end-state check.
- *False negative / double action:* if the right control was tapped but the predicate did not hold in time, a later route could tap **the same control again** (a toggle would flip back). Today nothing stops a later route re-proposing a control an earlier route already tapped (verified in the code). **Fix queued for after the protocol finishes: skip any control already acted on in this step.** Toggles are partly protected by `alreadySatisfied`/`satisfiedBy` preconditions.
- *Irreversible steps:* a tap that publishes cannot be undone before the predicate runs. So for `irreversible` intents the model may act only with confidence >= 0.8 and **only its first choice**; the assistant confirms with the user before publishing for someone else (`MOMO.md` rule 7). A confidently wrong first choice is still possible; a pre-action check (like V-Droid's verifier, section 5) is future work.
- *We never measured how often rules and model disagree directly.* We measured the consequence (wasted and silent wrong actions). A "model audits a rule's pick" ablation would measure disagreement itself: suggested future work.

2.3 "Who verifies the verifier?" (are the predicates right?)

- They are **hand-written** for 9 intents and exercised by a clean 9/9 run, the regression check, and the N=10 experiment (a wrong predicate would show up as failures on the healthy app).
- `feed.assert` is deliberately independent of every locator.
- On Android the oracle that judged the drift runs identifies the page by its **fragment name**, not by words, so a wrong-row tap cannot pass because the words changed (an earlier version used the activity name, which was the same for every row, and it was caught and fixed).
- Taught routines: the recorder infers a check from what changed on screen and **refuses a step it cannot verify**. `suggest` lets a model propose narrower checks, stored beside the original and applied only if the human replays with `--use-suggested`.
- **Weakness:** predicates are code we wrote; for open-ended tasks this does not scale. The literature on verifying GUI agents (section 5) is about exactly that gap.

2.4 "What if the page itself lies (stale or manipulated tree)?"

Our predicates read the live DOM/accessibility tree, so a lying tree can fool them. A 2026 study [V: arXiv 2607.04334] found that multimodal agents follow a stale structure over the pixels on up to 0.88 of probes,

with a self-recovery rate of at most 0.03, and that a **training-free consistency gate (compare pixels with structure)** was the only mitigation that reduced both hijacking and task error. Ours does not do pixel-vs-tree consistency checks; that is an honest gap and a good future-work item. Our vision rung exists but is not used to cross-check.

---

3. What a model can and cannot do in this system

- It sees a compact list of on-screen controls (tag, label, text, placeholder, rough position), **without ids or the typed caption**, and answers with at most 3 indices and a confidence.
- It cannot invent a control, cannot name one that is not on screen, and its answer is a *candidate* judged by the same check as every rule.
- Text on the page cannot widen that: the system prompt marks the list as untrusted data, and even if the model is fooled it can only point at something already on screen, which is then verified. **This limits harm; it does not make injection impossible**: planting text *can* make the model choose a wrong on-screen control (we observed the bare llm-only arm tap a planted control once), and the literature shows multi-step attacks that raise success rates (section 5).
- Reasoning ("thinking") tokens off was about 5x faster and half the cost with the same or better outcomes on the 3-config qualification, so it is the default.

---

4. Evidence: what is measured, and how strong it is

claim	result	strength
C1 recorded script vs ladder	N=10 per cell, 12 configs: ladder 100% [72-100] on clean, B1-B7 singly, B1+B2+B3+B4; recorded script 0% [0-28] on B1, B2, B6, B7 and every combination; both stop honestly on B8 and all-seven	**measured, N=10, no model**
C2/C3 model rung and verification	`qwen/qwen3.7-flash`, N=10, 5 drift configs x 5 arms (`results/published/semantic_n10_qwen37flash/`): rules alone 0% [0-28] on B1+B9, B1+B10+B9, B1+B10+B2+B9; **hybrid 100% [72-100]** on clean, B9, B1+B9, B1+B10+B9; both stop on the most extreme config; **llm-only 0% on every config with 1.0-4.4 silent wrong actions per run** (on the clean app it fails only the AI-label switch step, on all 10 flows, and reports success)	**real model, N=10, but repetitions are partly duplicates (temperature 0 + cache), so intervals are optimistic**
C6 real drift on Android	stock Settings, 5 rows x 2 sessions per cell: French: rules 2/10 = 20% [6-51], hybrid 10/10 = 100% [72-100]; large font: all arms 8/10 (two rows below the fold; nothing scrolls)	**real model, small n, one app**
C7 accessibility gap	2% of controls nameless across 5 screens x 3 sessions once labels of what is *inside* a clickable row count; looking only at the control's own attributes says 78% "anonymous"	**measured (stock Settings)**

claim	result	strength
C4/C5 planted instructions; distillation	built and tested with a stub; one real observation (n=1) for C4; the N=10 steps (`inject_n10`, `learn`) are running	**pending**

Say plainly: one mock web app, stock Android Settings, one real model so far, no user study, hand-written predicates, simulated (intent-side) drift in the Android suite except the language/font runs.

---

5. Prior art and novelty (be modest and precise)

5.1 What already exists

work	what it does	how this project differs
Healenium (EPAM) [V: GitHub README]	self-healing library for Selenium: when a locator fails it proposes healed locators (a configurable number of recovery tries, accepted only above a score cap) using data kept by its backend service	heals *test locators*; no intent, no outcome check of the healed tap, no user-facing failure message, no model
Zero-cost accessibility-tree self-healing, arXiv 2603.20358 (Mar 2026) [V]	a ten-tier ranked locator hierarchy extracted from the accessibility tree in one pass; re-extracts only broken selectors; no LLM; 31/31 tests passing on one site	**closest to our rules ladder.** Differences: we verify each action with a postcondition, add a verified model rung, stop honestly, and test under *injected and real drift* with a recorded-script baseline; their validation is one site with no drift comparison
V-Droid, arXiv 2503.15937 [V]	uses the LLM as a **verifier of candidate actions before** execution, not a generator; 59.5% on AndroidWorld, 4.3 s per step	their verifier is a learned judgement *before* acting; ours is a deterministic check of the *observed result after* acting. They are complementary (a pre-action verifier would help our irreversible steps)
Minitap, arXiv 2602.07787 (Feb 2026) [V]	100% on AndroidWorld; ablation: **deterministic post-validation of text input against device state adds +7 points**	independent support for our core idea (check the result, do not trust that the tap worked). They solve the benchmark; we study robustness to drift and honest failure
VAGEN (2602.00575), MagicGUI-RMS (2601.13060) [V]	verifier/reward models for GUI agents (LLM-as-judge limits: partial observability)	they judge whole trajectories for training; ours are cheap per-step postconditions at run time

work	what it does	how this project differs
AppAgent (2312.13771), **SeeAct** (2401.01614), **UI-TARS** (2501.12326), **Set-of-Mark** (2310.11441) [V]	LLM/VLM agents that act from screenshots or UI trees; set-of-marks overlays numbered tags (we reuse the idea for our vision rung)	generators; no per-step verified-candidate contract or honest-failure behaviour; no head-to-head against them here
Computer-use agents for blind users, arXiv 2609.00524 (Sep 2026) [V]	3-week diary study, 8 blind users, 1,258 commands, 12 desktop apps: best model (GPT-5) **52.5%** success; failures in grounding, planning, constraint tracking, termination	shows the problem is real and unsolved, and that *termination* (knowing when it failed) matters, which is our honest-failure focus
MCP accessibility layer, arXiv 2608.24898 (Jul 2026) [V]	proposes MCP as a uniform layer over accessibility trees for screen-reader agents; **conceptual, no implementation or evaluation**	we implement and evaluate an accessibility-tree layer; exposing our tool through MCP is future work
Prompt-injection benchmarks: WebInject, WAInjectBench, WebSentinel, SnapGuard, MUZZLE, VPI-Bench, OS-Harm, OSGuard, **StepJack** (Aug 2026), WASP, AgentDojo [V]	attacks and defenses for web/computer-use agents; StepJack: multi-step attacks raised success by up to 31.2 points (GPT-5.4-mini 41.7% to 72.9%)	we test a small direct version (B11) and rely on structure (index-only answers + verification + confidence gate), not detection. We do not claim robustness to multi-step attacks
"Do mobile agents need the phone screen?", arXiv 2606.19388 (Jun 2026) [V]	CLI/API agents beat GUI baselines on AndroidWorld/MobileWorld where an API exists (Claude Code 71.8% vs 69.3% best reproducible GUI baseline)	supports the poster's "post-MCP" thesis: use APIs where they exist; GUI is the fallback for apps without them (our case)
Commercial computer-use / browser agents (Anthropic computer use, OpenAI Operator / ChatGPT agent, Google Mariner / Gemini) **[M, not re-verified]**	closed generators that act from screenshots	we do not compete on breadth; see `SYNTHIE.md` section 6 for a (self-reported, unverified) comparison. Volatility example [V]: OpenAI's ChatGPT Atlas browser is already listed as discontinued (Wikipedia, 2026-09-29)
Testim, mabl, Applitools, Tricentis [M, not re-verified]	commercial AI-assisted test automation with self-healing locators	test tooling for QA teams; not an assistant for users who cannot take over

work	what it does	how this project differs
Benchmarks: AndroidWorld (116 tasks, 20 apps) [V], MobileWorld (201 tasks, 20 apps; best framework 51.7%, end-to-end 20.9%) [V]	AndroidWorld is saturated (>90%; Minitap 100%); MobileWorld is the harder successor	we did not run them: our question is robustness under UI change and honest failure, which they do not vary. Running a drifted MobileWorld subset is future work

5.2 What is NOT novel (never claim these)

Self-healing locators; LLMs choosing UI elements; set-of-marks vision prompting; verification as an idea; prompt-injection awareness; measuring accessibility trees.

5.3 What the contribution actually is (claim this, modestly)

1. **A verified-candidate contract applied uniformly**: rules, memory, model and geometry are all just *proposers*, and one postcondition decides, so a model can be added without being trusted.
2. **Honest failure as a designed output** for users who cannot take over, specified in the assistant contract (`MOMO.md`), with the success/failure meaning of `ok` defined as "every predicate held and the result is visible".
3. **Measurement design**: silent-wrong-action and planted-instruction counts as first-class metrics; arms that separate what rules, verification and the model each add; real drift (system language, font) alongside injected drift; reports that carry their model, N and validity.
4. **A correction to a common measurement**: counting a control as "anonymous" from its own attributes overstates the accessibility gap (78% vs 2% on stock Settings).
5. **Remembered model answers** (distillation) to cut model cost; real-model evidence still pending. It is an **engineering and evaluation contribution, not a new algorithm.** Say that before anyone else does.

---

6. The question bank (short answers; evidence pointer in brackets)

Problem and motivation

1. **Why not just use APIs or MCP?** Use them wherever they exist [V: MCP adopted by OpenAI and Google, donated to the Linux Foundation, Dec 2025]. Many apps (the poster's local marketplaces) have none, and a 2026 CLI-vs-GUI study shows APIs/CLIs beat GUI where available [V 2606.19388]. GUI automation is the fallback for the rest.
2. **Why is this an accessibility project?** The same data a screen reader gets (the accessibility tree) is our measuring instrument; a control with no name is a finding for both. And the target user cannot take over a failed step, so *silent* failure is the harm [plan 2.1-2.2].
3. **Is the problem real?** In a 2026 diary study with blind users the best computer-use model succeeded 52.5% of the time across 1,258 commands [V 2609.00524].
4. **Who is the user and did you test with them?** Users who cannot operate the app themselves. **We did not run a user study**; say so and name it as the main future work.

Design

5. **How does it know a step worked?** Section 2.1: a postcondition on the next screen, independent of the route.
6. **Rules and model disagree?** Section 2.2: no voting; the screen decides; wasted taps are counted; known gap (same control retried) and its queued fix.
7. **Why a ladder and this order?** Strongest, cheapest, most stable first; each route must match exactly one control; `learned` and `semantic` sit before position because a blind positional guess can tap something that changes the screen (we measured this: it derailed a flow before the reorder).
8. **Why must a route match exactly one control?** It is what prevents wrong-element taps from ambiguous locators (the "shared id" failure, B4).
9. **What is the model shown?** Section 3.
10. **Why not give the model a screenshot?** The llm-only arm is that idea without checks; it made silent wrong actions. A screenshot with numbered badges is our vision rung for controls with no readable name (tested at n=1; N=5 in the protocol).
11. **Why not a vision-only agent like UI-TARS?** [V 2501.12326] Those need no tree, which is a strength where no tree exists. Where a tree exists it gives exact names and states cheaply; we keep vision as a rung. No head-to-head was run.
12. **What does remembering do?** It records which route worked and turns a model answer into a locator with no id, so the model is asked once per UI change, not once per run (stub: 7 to 1 calls; real-model result pending).

Safety and correctness

13. **A wrong tap on an irreversible step?** Confidence >= 0.8 and first choice only for `irreversible` intents; the assistant confirms before publishing for someone else; a pre-action check is future work.
14. **Prompt injection?** Section 3 and C4: structural limits, one observation, N=10 pending, no claim against multi-step attacks [V StepJack].
15. **What if the model is confidently wrong?** The post-check catches it for reversible steps; for irreversible ones see 13.
16. **Temperature, nondeterminism?** Temperature 0, seed 0, cache; repetitions measure the app, not sampling variance (say this unprompted).
17. **Does data leave the machine?** Control labels and positions from our mock app and the emulator's stock Settings go to OpenRouter (free models may log prompts); no personal data, accounts or passwords; keys are git-ignored and scrubbed from output.

Evaluation

18. **Your breaks are self-made.** Yes for the web (controlled stress tests). The Android language/font runs change the app underneath unchanged intents.
19. **Sample sizes?** C1 and the model drift experiment (C2/C3) are N=10 with Wilson intervals. But with temperature 0, a cache and identical app states, the ten repetitions are **not ten independent samples of the model**: they show variation in the app and harness only. Say this before being asked. The planted-instruction, vision and distillation steps are still running; Android is n=10 intents per cell. 20a. **Why does the bare model fail even on the clean app?** In the N=10 run it fails exactly one step, `setting.toggle` (the 'Add AI label' switch), on every flow, and publishes the post anyway: it reports success while a step was silently skipped. Everything else it does correctly. So the honest statement is not 'a model cannot do this task' but 'without a check, one wrong pick goes unnoticed'. (Most likely it

taps the label row instead of the actual switch; the call log for this step was not kept, so say 'most likely'.)

20. **Why does llm-only equal hybrid on Android?** Settings shows about 9 controls, an easy case; verification matters on the 60-control web app and with planted instructions. Do not use Android to argue for verification.
21. **Is the baseline fair?** It is a recorded-locator script made as a normal recorder would (an earlier version used an absolute path for a text-only control and failed on the clean app: that was found and fixed; quote only the fixed numbers).
22. **Why not AndroidWorld/MobileWorld?** AndroidWorld is saturated [V], MobileWorld is the harder successor [V]; neither varies UI drift or scores honest failure. A drifted subset is future work.
23. **"9/9 under chaos"?** Only for single breaks and small combinations; all-seven and B8 stop honestly. See section 0.
24. **Threats to validity?** One mock app, one real model, hand-written predicates, one emulator image, no scrolling, no user study.

Novelty

25. **What is novel?** Section 5.3, modestly. **Healenium/Testim already self-heal.** They heal test locators; we verify outcomes, add a verified model, fail honestly. **Isn't the 2026 accessibility-tree self-healing paper the same?** It is the closest to our ladder (section 5.1): no postcondition checks, no model rung, one site, no drift comparison.
26. **V-Droid already uses verifiers.** Before acting, with a learned judge; ours checks the observed result after acting, deterministically. Complementary.
27. **Why is this enough for an FYP?** A working, tested system, an honest evaluation design, real-device evidence and clearly stated limits; claims are sized to the evidence.

Scope and ethics

28. **The poster says fingerprint stealth and touch telemetry.** Section 0, item 1: out of scope by decision; we measured protection behaviour and did not build evasion.
29. **Daraz/OLX?** Section 0, item 2: motivating class, not evaluated.
30. **Ethics?** The harness is an overlay on our own mock; no real accounts; `protect` measures a site we control and never solves or spoofs a challenge; the ethics paragraph is plan 11.4.

Android

31. **Why an emulator, not Instagram?** Instagram needs a login and has bot protection (out of scope); stock Settings is real, reproducible and allowed.
32. **What did Android show?** 2% nameless controls once child labels count; French drift 20% to 100% with the model rung; large font exposes missing scrolling.
33. **Why does the large font fail?** Two rows fall below the fold and nothing scrolls yet; the accessibility tree says which container scrolls, so this is the next fix.

Assistant, Momo, Kenna, synthie

34. **What is Momo?** The assistant governed by `agent/MOMO.md`: goals not locators, honest failure, no blind retry, confirm before publishing for someone else.

35. **What is a synthie?** A long-running agent with its own host, identity, typed tools and logs; it is the kind of agent that calls this tool. Most descriptions are a synthie's self-report [SYNTHIE.md section 0]: verify before slides.
36. **Where does MCP fit?** The tool already speaks one JSON document per call with a manifest; wrapping it as an MCP server is a short step and future work. A 2026 paper proposes MCP for accessibility trees but only conceptually [V 2608.24898].

Hostile questions

37. **"Isn't this Selenium with extra steps?"** Selenium is the driver; the contribution is intent + verified candidates + honest failure + measurement under drift.
38. **"Show it failing."** `fyp.mjs run --breaks B8` (an unreachable control) stops at step 1 with the plain-words message and a screenshot.
39. **"Why trust a cheap model?"** On purpose: if verification works with a cheap model it matters more than model strength. A second model is still to do.
40. **"What would you do with another month?"** User study with screen-reader users; scrolling; skip already-tried controls; pre-action verification for irreversible steps; pixel-vs-tree consistency gate; a second model and app; a drifted MobileWorld subset; an MCP wrapper.

---

7. Numbers to memorise

- C1: ladder 100% [72-100] vs recorded script 0% [0-28] on B1/B2/B6/B7 (N=10); both stop honestly on B8.
- Android French: rules 2/10 = 20%, hybrid 10/10 = 100% (real model, 5 rows x 2 sessions).
- Nameless controls: 2% (name-aware) vs 78% (own attributes).
- N=10 real model: hybrid 100% [72-100] on 3 drifted configs where rules score 0%; llm-only 0% everywhere (clean: fails only the AI-label step). Qualification: thinking off 1.7 s vs 6 s median, about $0.00007 per call. Cost of everything so far (qualification, Android, 1,050-call N=10 step): **$0.059**.
- Related studies [V]: blind-user CUA study 52.5%; AndroidWorld saturated (Minitap 100%); MobileWorld best framework 51.7%.

8. Queued improvements (after the real-model protocol finishes, so the evidence set stays on one harness version)

1. Skip any control already acted on within a step (closes the double-action gap, section 2.2).
2. Scrolling step using the accessibility tree's scrollable container.
3. Optional pre-action check for `irreversible` intents.
4. Pixel-vs-tree consistency gate.
5. A "model audits a rule's pick" ablation to measure rule/model disagreement directly.
6. Update the poster (section 0).

9. Demo checklist

Emulator window on screen (about 340 x 760 at the right edge) - cache path for offline model steps (`DEFENSE.md`) - `FYP_SEMANTIC_REASONING=off` set - backup video recorded - key files not open on screen (OneDrive-synced) - run `python Kenna/tools/check_links.py` and the regression check once.

10. If something breaks live

Model call fails: say "this run is marked invalid by design", rerun from the cache. Emulator frozen: cold boot with `-no-snapshot-load` (see `NEXT_STEPS.md` section 2). Network down: steps 3-5 of the demo reuse cached model answers.

11. Sources (accessed 2026-10-29)

arXiv API (abstracts quoted): 2603.20358 zero-cost accessibility-tree self-healing; 2503.15937 V-Droid; 2602.07787 Minitap; 2602.00575 VAGEN; 2601.13060 MagicGUI-RMS; 2609.00524 CUAs for blind users; 2608.24898 MCP accessibility tree; 2607.04334 pixels vs structure; 2606.19388 CLI vs GUI agents; 2512.19432 MobileWorld; 2405.14573 AndroidWorld; 2608.06477 StepJack; 2505.11717 WebInject; 2510.01354 WAInjectBench; 2602.03792 WebSentinel; 2604.25562 SnapGuard; 2602.09222 MUZZLE; 2506.02456 VPI-Bench; 2506.14866 OS-Harm; 2606.15034 OSGuard; 2406.13352 AgentDojo; 2504.18575 WASP; 2312.13771 AppAgent; 2401.01614 SeeAct; 2501.12326 UI-TARS; 2310.11441 Set-of-Mark; 2408.06224 grey-literature review of AI-assisted test automation. Links: `https://arxiv.org/abs/<id>`. Wikipedia REST summaries: Model Context Protocol (2026-10-29), ChatGPT Atlas (2026-09-29). GitHub: `healenium/healenium-web` README; `google-research/android_world` README. OpenRouter public model listing for `qwen/qwen3.7-flash`. **Not verified in this session [M]:** product claims about Anthropic computer use, OpenAI Operator / ChatGPT agent, Google Mariner / Gemini computer use, Testim, mabl, Applitools. Check their current documentation before citing.